

10-bit SAR ADC Design in Sky130 – Project Report

Contents

1	Introduction and Specifications	2
2	Architecture Overview	2
3	Block Design and Sizing	3
3.1	Python Behavioral Model	3

1 Introduction and Specifications

The successive-approximation (SAR) ADC is the most commonly deployed topology of analog-to-digital converter. Owing to its low power consumption, low latency and compact form factor, it has proven supremely versatile, with notable usage in IoT devices, medical equipment and microcontrollers. This document describes an original 10-bit SAR ADC designed and verified in the SkyWater Sky130 open-source 130nm process — a CMOS PDK developed by SkyWater Technology and Google, providing full device model access without NDA — targeting the specifications in Table 1. The design covers the complete front-end flow: behavioral modeling from first principles, transistor-level schematic capture in xschem, ngspice simulation against Sky130 device models, and Verilog RTL synthesized to Sky130 standard cells via Yosys. Physical layout is out of scope; the project targets schematic-level verification.

Table 1: Design Specifications

Parameter	Value
Resolution	10 bits
ENOB	≥ 9 bits
Supply Voltage, V_{DD}	1.8 V
Input Range	0 – 1.8 V
Process	SkyWater Sky130, 130nm

The design followed a bottom-up flow, progressing sequentially through each of the following stages:

1. First-principles sizing and yield analysis via Python behavioral model
2. Block-level schematic capture in xschem and isolation verification in ngspice
3. Verilog RTL synthesis to Sky130 standard cells via Yosys
4. Full-ADC SPICE integration and static performance characterisation

This document reports the design decisions, block-level verification, integration debugging and static characterisation of the 10-bit SAR ADC. Ultimately, the design achieves a static ENOB of 7.27 bits, falling short of its 9-bit target, attributable to a characterised top-plate feedthrough mechanism, detailed in Section ??..

2 Architecture Overview

During the sampling phase, the bootstrapped switch connects V_{in} to all bottom plates of the CDAC array simultaneously, while the top plate is reset to $V_{cm} = V_{ref}/2$ via a transmission gate. This establishes a charge on the top plate node of $Q = (V_{ref}/2 - V_{in}) \cdot C_{total}$.

At the start of conversion, the bottom plates disconnect from V_{in} and switch to ground, leaving the top plate floating with its charge preserved. The SAR logic then performs a binary search from the MSB downward: each trial bit steers its corresponding bottom plate to V_{ref} , shifting V_{DAC} upward by an amount proportional to that capacitor's weight. The StrongARM comparator evaluates whether V_{DAC} has crossed $V_{ref}/2$, equivalent to testing whether V_{in} exceeds the analog value of the current trial code. The SAR logic retains or clears each bit accordingly:

1. Assert trial code 1000000000; bottom plate of $512C$ switches to V_{ref} , shifting V_{DAC} upward
2. If $V_{DAC} < V_{ref}/2$ (equivalently, $V_{in} > V_{ref}/2$), retain bit 9; otherwise clear it and return that bottom plate to ground
3. Assert the next trial bit and repeat in descending order until all 10 bits are resolved

This process is summarized by the block diagram in figure 1.

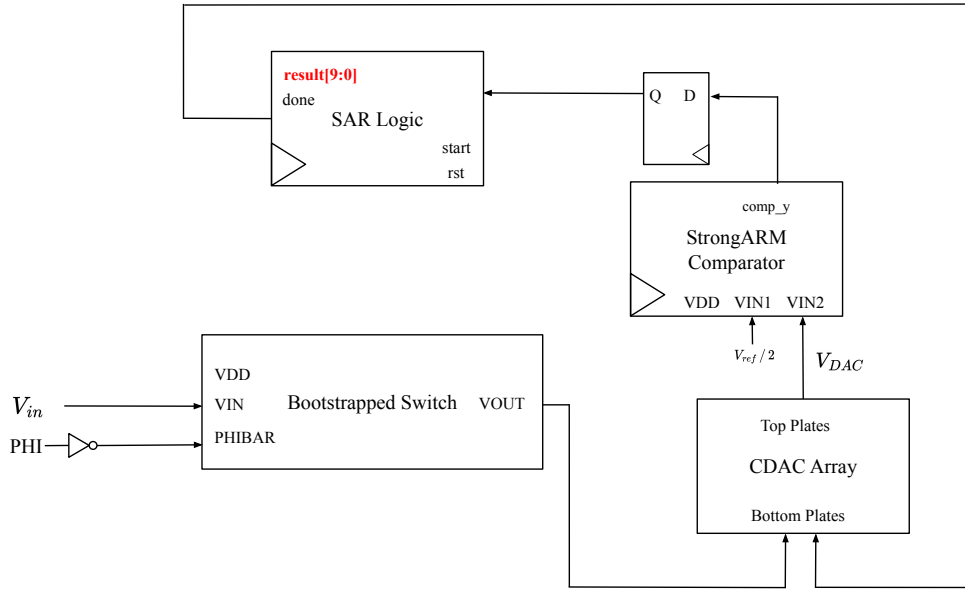


Figure 1: Block diagram of SAR ADC

3 Block Design and Sizing

3.1 Python Behavioral Model

The minimum valid unit capacitance, C_{unit} , that can be used in the CDAC array is governed by the thermal noise floor in equation 1.

$$C_{min} = \frac{kT}{V_{LSB}^2} \quad (1)$$

with $V_{LSB} = \frac{V_{DD}}{N}$, where N is the resolution of the instrument. Substituting $k = 1.38 \times 10^{-23} \text{ J} \cdot \text{K}$, $T = 300 \text{ K}$, $V_{DD} = 1.8 \text{ V}$ and $N = 10$, $C_{min} = 5.36 \text{ fF}$.

In order to determine a suitable unit capacitance, a Monte Carlo mismatch analysis was conducted. In each trial, C_{unit} was swept from 5 fF to 54 fF and a binary-weighted capacitor array, $[512, 256, 128, \dots, 2, 1, 1] \times C_{unit}$ was generated, representing the CDAC array. Each capacitor in this array was perturbed according to $\sigma_{C/C} / \sqrt{n_{cells}}$, where the mismatch $\sigma_{C/C}$ is 2%, and $n_{cells} = C_{unit} / C_{min}$. Using this representation of the CDAC array, the SAR conversion algorithm was executed on $2^{16} = 65\,536$ input voltages, 64 per binary code. For each trial, the DNL and INL were computed and the maximum INL was recorded. This process was repeated for 10 000 trials.

The yield is computed as the fraction of trials where the maximum INL is less than 0.5 LSB. As can be seen in figure 2, the yield surpasses its target of 99% at approximately 24 fF. Therefore, to allow for some margin, a target value of $C_{unit} = 25 \text{ fF}$ was selected, giving a total CDAC capacitance of $1024 \times 25 \text{ fF} = 25.6 \text{ pF}$.

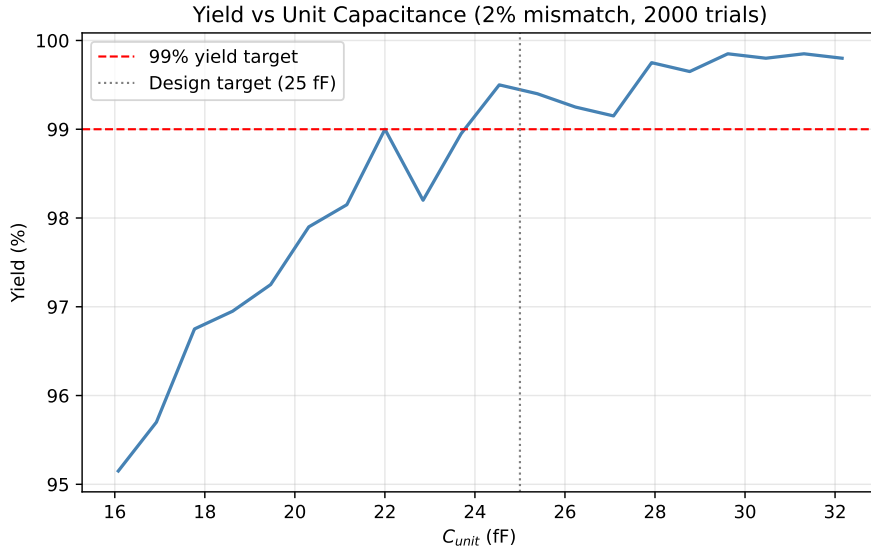


Figure 2: Yield vs. C_{unit} sweep across 10 000 Monte Carlo trials at $\sigma_{C/C} = 2\%$.

To validate the selected $C_{unit} = 25 \text{ fF}$, a further 10 000-trial Monte Carlo simulation was run at this fixed value. Figure 3 shows the resulting distribution of maximum INL across all trials. The distribution is centred near 0.2 LSB, with a tail that barely reaches the 0.5 LSB threshold, confirming a yield of 99.2%.

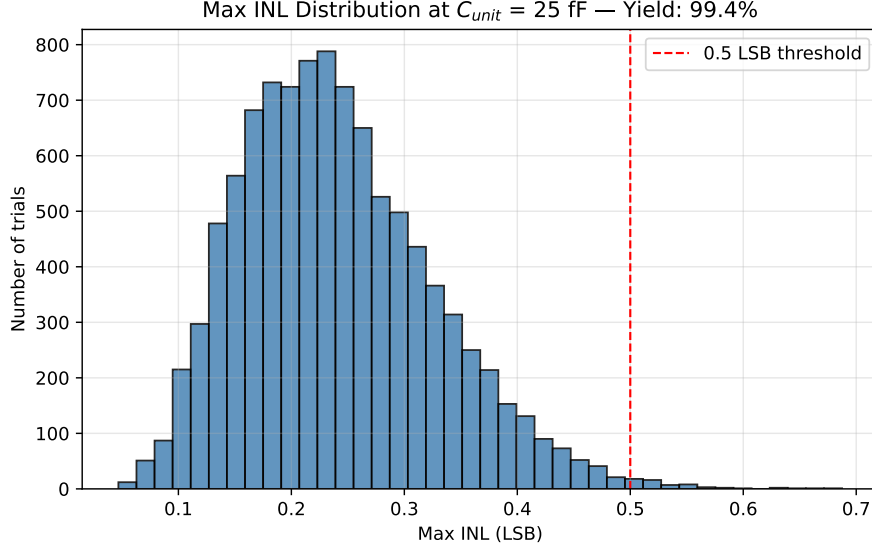


Figure 3: Distribution of maximum INL at $C_{unit} = 25$ fF, 10 000 Monte Carlo trials, $\sigma_{C/C} = 2\%$. Yield: 99.2%.

Figure 4 shows the DNL and INL of a representative single trial at $C_{unit} = 25$ fF. The DNL remains well within ± 0.5 LSB across all 1024 output codes, with no missing codes. The INL exhibits a random-walk character, as expected from independent capacitor mismatch errors accumulating across the code range, and stays within ± 0.5 LSB throughout.

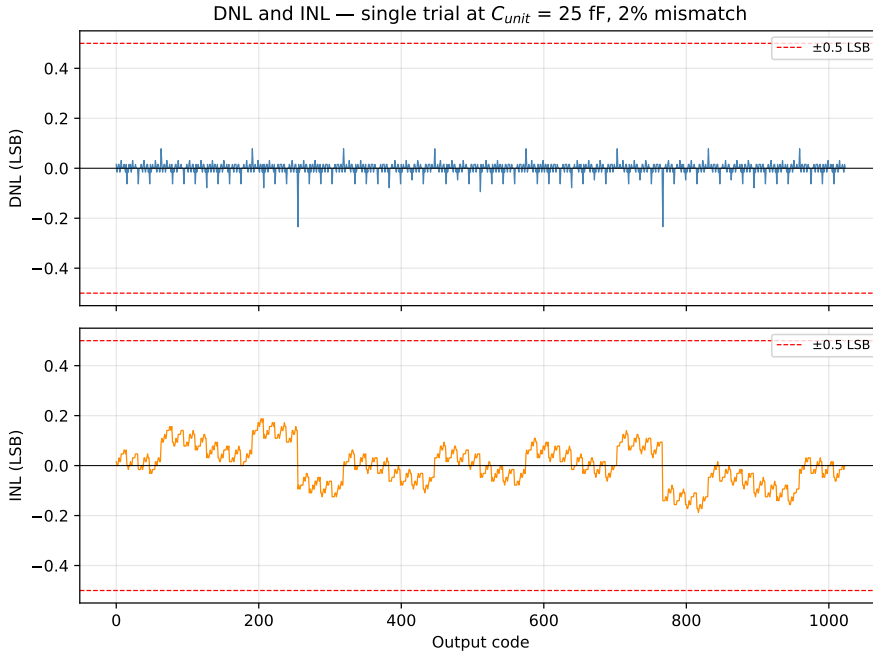


Figure 4: DNL and INL for a single representative trial at $C_{unit} = 25$ fF, $\sigma_{C/C} = 2\%$.